

DAYANANDA SAGAR UNIVERSITY

Devarakaggalahalli, Harohalli
Kanakapura Road, Bengaluru South Dt - 562112, Karnataka, India



**SCHOOL OF
ENGINEERING**

**Bachelor of Technology
in
COMPUTER SCIENCE AND ENGINEERING**

Major Project Phase-II Report

**Intellectual Smart Hybrid AI System for Real-Time Threat Intelligence in the
Internet**

Batch: 89

By

**NU Chinnappa - ENG22CS0377
Sriram Puranik - ENG22CS0467
Tejas KM - ENG22CS0480
Kalyan Gogoi - ENG23CS1031**

Under the supervision of

**Dr. Naitik S T
Assistant Professor, CSE**

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING,
SCHOOL OF ENGINEERING
DAYANANDA SAGAR UNIVERSITY,**

(2025-2026)

DAYANANDA SAGAR UNIVERSITY



**SCHOOL OF
ENGINEERING**

Department of Computer Science & Engineering

Devarakaggalahalli, Harohalli, Kanakapura Road, Bengaluru South Dt - 562112
Karnataka, India

CERTIFICATE

This is to certify that the Major Project Stage-II work titled **“Intellectual Smart Hybrid AI System for Real-Time Threat Intelligence in the Internet”** is carried out by **Sriram Puranik (ENG22CS0467)**, **NU Chinnappa (ENG22CS0377)**, **Kalyan Gogoi (ENG23CS1031)**, **Tejas Km (ENG22CS0480)**, bonafide students seventh semester of Bachelor of Technology in Computer Science and Engineering at the School of Engineering, Dayananda Sagar University, Bangalore in partial fulfillment for the award of degree in Bachelor of Technology in Computer Science and Engineering, during the year **2025-2026**.

Guide Name

Assistant/Associate/ Professor
Dept. of CS&E,
School of Engineering
Dayananda Sagar University

Date:

Dr. Girisha G S

Chairman CSE
School of Engineering
Dayananda Sagar University

Date:

Dr. Udaya Kumar Reddy K R

Dean
School of Engineering
Dayananda Sagar
University

Date:

Name of the Examiner

- 1.
- 2.
- 3.

Signature of Examiner

DECLARATION

We, **Sriram Puranik (ENG22CS0467)**, **NU Chinnappa (ENG22CS0377)**, **Kalyan Gogoi (ENG23CS1031)**, **Tejas Km (ENG22CS0480)**, are students of the seventh semester B.Tech in **Computer Science and Engineering**, at School of Engineering, **Dayananda Sagar University**, hereby declare that the Major Project titled “**Intellectual Smart Hybrid AI System for Real-Time Threat Intelligence in the Internet**” has been carried out by us and submitted in partial fulfilment for the award of degree in **Bachelor of Technology in Computer Science and Engineering** during the academic year **2025-2026**.

Student

Signature

Name1: Sriram Puranik

USN : ENG22CS0467

Name2: Tejas KM

USN : ENG22CS0480

Name3: NU Chinnappa

USN : ENG22CS0377

Name4: Kalyan Gogoi

USN : ENG23CS1031

Place : Bangalore

Date :

ACKNOWLEDGEMENT

It is a great pleasure for us to acknowledge the assistance and support of many individuals who have been responsible for the successful completion of this Special Topic.

First, we take this opportunity to express our sincere gratitude to School of Engineering & Technology, Dayananda Sagar University for providing us with a great opportunity to pursue our Bachelor's degree in this institution.

We would like to thank **Dr. Uday Kumar Reddy K R, Dean, School of Engineering, Dayananda Sagar University** for his constant encouragement and expert advice. It is a matter of immense pleasure to express our sincere thanks to **Dr. Girisha G S, Chairman, Department of Computer Science, and Engineering, Dayananda Sagar University**, for providing the right academic guidance that made our task possible.

We would like to thank our guide **Dr. Naitik S T Assistant Professor, Dept. of Computer Science and Engineering, Dayananda Sagar University**, for sparing her valuable time to extend help in every step of our project work, which paved the way for smooth progress and the fruitful culmination of the project.

We would like to thank our **Project Coordinators Dr. Sivananda Lahari Reddy, Dr. Kumar Dilip and Dr. Naresh P** as well as all the staff members of Computer Science and Engineering for their support.

We are also grateful to our family and friends who provided us with every requirement throughout the course.

We would like to thank one and all who directly or indirectly helped us in the Project work.

TABLE OF CONTENTS

LIST OF FIGURES.....	8
ABSTRACT	8
CHAPTER 1 INTRODUCTION.....	9
CHAPTER 2 PROBLEM DEFINITION	10
CHAPTER 3 LITERATURE SURVEY	11-13
CHAPTER 4 PROJECT DESCRIPTION	14-15
CHAPTER 5 REQUIREMENTS	16-20
CHAPTER 6 METHDOLOGY.....	21-23
CHAPTER 7 EXPERIMENTATION	24-30
CHAPTER 8 RESULT	31-40
CHAPTER 9 DELIVERABLES	41-43
REFERENCES.....	44-46

NOMENCLATURE USED

Acronym/Term	Full Form / Definition
IS-HAITI	Intellectual Smart Hybrid AI System for Real-Time Threat Intelligence in the Internet
GNN	Graph Neural Network
GCN	Graph Convolutional Network
GAT	Graph Attention Network
SAGE	GraphSAGE (Inductive representation learning on large graphs)
SMOTE	Synthetic Minority Over-sampling Technique
X-TIS	Explainability & Transparency Intelligence System
CADA	Continuous Adaptation & Defense Architecture
TCN	Temporal Convolutional Network
C2	Command & Control
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
SOC	Security Operations Center
APT	Advanced Persistent Threat
XAI	Explainable Artificial Intelligence
DLP	Data Loss Prevention
EDR	Endpoint Detection & Response
NSL-KDD	NSL-KDD Network Intrusion Detection Dataset
CICIDS	Canadian Institute for Cybersecurity Intrusion Detection System Dataset
PCAP	Packet Capture file format
CSV	Comma-Separated Values
ROC-AUC	Receiver Operating Characteristic - Area Under the Curve
F1-Score	Harmonic mean of Precision and Recall
MFA	Multi-Factor Authentication
DNS	Domain Name System

LIST OF FIGURES

Fig. No.	Description of the figure	Page No.
1	Fig 1: High-Level Workflow of the IS-HAITI Threat Detection System	23
2	Fig 2: GNN-Based Service Node Attack Classification	33
3	Fig 3: ROC Curve for GNN-Based Attack Detection	34
4	Fig 4: High-Risk Services Identified by the Proposed Model	35
5	Fig 5: IRC Subgraph Highlighting Attack Relations	36
6	Fig 6: NetBIOS-SSN Attack Relationship Graph	38
7	Fig 7: Pump-and-Dump Attack Graph – Data Exfiltration Path	39
8	Fig 8: NetBIOS-DGM Attack Subgraph	40

ABSTRACT

Modern cyber threats increasingly exploit multiple network services, communication channels, and web platforms, making traditional single-model intrusion detection systems insufficient for comprehensive analysis. **The IS-HAITI (Intellectual Smart Hybrid AI Threat Intelligence)** framework addresses this challenge by combining behavioral learning, structural analysis, and explainable intelligence within a unified cybersecurity system.

The implemented system performs threat analysis using benchmark intrusion detection datasets including **KDD**, **CICIDS2017**, and **UNSW-NB15**. A tabular Transformer architecture is employed to learn behavioral patterns from network flow features and generate probabilistic assessments of malicious activity. These behavioral predictions are further analyzed at the service level, where network services are modeled as interconnected entities. Graph-based reasoning enables identification of services associated with coordinated or abnormal behavior patterns.

The framework integrates learned model outputs with rule-supported analysis and produces structured detection reports together with explainability outputs through the Explainable Threat Intelligence System (X-TIS). The system demonstrates capability in identifying phishing and spam communications, malicious URLs and credential harvesting websites, network reconnaissance activities, brute-force authentication attempts, service enumeration patterns, and botnet-related communication behavior.

CHAPTER 1 INTRODUCTION

The increasing complexity of cyber-attacks has created significant challenges for traditional network security systems. Modern attackers rarely rely on a single exploit instead, attacks typically involve multiple stages such as credential attacks, malicious communication establishment, and persistence mechanisms. These activities often appear normal when analyzed independently but reveal malicious intent when behavioral and structural relationships are considered together.

Network intrusion datasets contain detailed information about communication services, protocols, connection statistics, and traffic behavior. Services such as HTTP, FTP, Telnet, SSH, SMTP, and IRC frequently act as operational points through which attacks propagate. Therefore, analyzing cyber threats at the service level provides a more meaningful understanding of malicious activity compared to isolated packet classification.

Our project framework focuses on hybrid cyber threat analysis by combining behavioral learning models and service-level structural reasoning. The system learns attack behavior from multiple intrusion detection datasets and evaluates how risk propagates across interacting services. Instead of replacing traditional approaches, the framework integrates learning-based intelligence with interpretable analysis outputs.

CHAPTER 2 PROBLEM DEFINITION

Modern cyber-attacks are increasingly distributed across multiple network services and communication channels, making them difficult to detect using traditional intrusion detection systems that rely on signature matching or independent record classification. Individual network connections may appear normal when analyzed in isolation, even though they collectively represent activities such as reconnaissance, brute-force attempts, phishing communication, or coordinated malicious behavior. Existing machine learning approaches improve detection capability but often fail to capture relationships between services and lack interpretability required for practical security analysis. Therefore, the problem addressed in this project is to develop a cyber threat detection framework capable of learning behavioral patterns from network data, identifying attack-linked services through structural analysis, and producing interpretable threat intelligence outputs that assist in understanding and investigating malicious activity.

CHAPTER 3 LITERATURE REVIEW

- **AI-Driven Transformer Frameworks for Network Anomaly Detection:** Kim et al. proposed a Transformer-based framework capable of performing real-time anomaly detection in network traffic by learning complex feature relationships using attention mechanisms. The model demonstrated improved detection capability compared to traditional classifiers due to its ability to capture long-range dependencies within network flows. However, the study highlighted that Transformer models alone lack contextual understanding of network relationships, motivating integration with structural analysis methods similar to those used in the IS-HAITI framework. [1]
- **Graph Neural Networks Embedded with Domain Knowledge:** Zhou et al. introduced a graph neural network approach enhanced with domain knowledge for network threat intelligence generation. By representing entities as nodes and relationships as edges, the system successfully captured coordinated attack behaviour. The work emphasized that combining learned features with graph reasoning improves detection of complex attacks such as lateral movement and reconnaissance, directly aligning with the service-level graph reasoning adopted in IS-HAITI. [2]
- **DeepGuard Transformer-Based Cyber Threat Detection:** Demir and Koc developed DeepGuard, a Transformer-based real-time threat detection framework that learns behavioral patterns from network data streams. Their results demonstrated high detection accuracy for evolving threats; however, the system required complementary mechanisms to interpret decisions and analyze structural dependencies, supporting the hybrid analytical design implemented in the IS-HAITI system. [3]
- **Hybrid Deep Learning Threat Intelligence for Malware Detection:** Singh et al. presented a hybrid deep learning architecture combining multiple neural models for malware threat intelligence. The hybrid design improved detection performance while reducing false positives compared to standalone deep learning models. The study

Intellectual Smart Hybrid AI System Real-Time Threat Intelligence in the Internet

reinforced the importance of combining multiple intelligence sources, which forms the core principle behind the hybrid IS-HAITI framework. [4]

- **Explainable Artificial Intelligence for Cyber Threat Intelligence:** Miller and Brown explored explainable AI techniques for cybersecurity analytics and demonstrated that interpretable models significantly improve analyst trust and investigation efficiency. The study emphasized the need for explanation-driven detection systems, influencing the Explainable Threat Intelligence System (X-TIS) incorporated within IS-HAITI. [5]
- **Federated Learning for Collaborative Cybersecurity Intelligence:** Li et al. proposed a federated learning approach allowing distributed systems to collaboratively learn threat patterns without sharing sensitive data. The work highlights future directions for scalable and privacy-preserving threat intelligence systems and suggests possible extensions of IS-HAITI toward decentralized intelligence sharing environments. [6]
- **Hybrid TCN–Transformer Insider Threat Detection:** Park et al. combined Temporal Convolutional Networks and Transformer architectures for insider threat detection using behavioral sequences. The hybrid temporal modeling approach improved detection of subtle malicious behavior occurring over time. This supports the adoption of Transformer-based behavioral learning in IS-HAITI for identifying evolving threat patterns. [7]
- **Multimodel Deep Learning for Online Threat Detection:** Ahmed and Rahman proposed a multimodel deep learning framework integrating multiple neural architectures for continuous online threat monitoring. Their results demonstrated improved robustness against diverse attack scenarios, validating the effectiveness of hybrid analytical systems similar to the IS-HAITI design philosophy. [8]
- **GNN-Based DoS Detection in Software-Defined Networks:** Sun et al. applied graph neural networks within software-defined networking environments to detect denial-of-service attacks. Graph representations enabled identification of coordinated traffic anomalies across network nodes, reinforcing the importance of structural learning mechanisms adopted in IS-HAITI service-level risk analysis. [9]

- **Decentralized Threat Intelligence Using Federated Learning:** Gupta and Patel introduced decentralized cybersecurity intelligence models where multiple systems collaboratively detect threats. Their findings suggest improved scalability and resilience in threat detection architectures, indicating potential expansion directions for hybrid intelligence frameworks such as IS-HAITI. [10]
- **Transformer-Based Malware Detection from System Logs:** Wang et al. utilized Transformer architectures to analyze system process logs for malware detection. The attention mechanism enabled identification of complex behavioral relationships among system events, demonstrating the effectiveness of Transformer models in cybersecurity behavior analysis tasks relevant to IS-HAITI. [11]
- **NSL-KDD Dataset for Intrusion Detection Evaluation:** Moustafa and Slay evaluated anomaly detection systems using improved KDD dataset variants designed to address redundancy issues present in earlier benchmarks. The dataset provides structured network service information used for behavioral learning experiments in IS-HAITI. [12]
- **CICIDS2017 Intrusion Detection Dataset:** Sharafaldin et al. developed CICIDS2017 to provide realistic enterprise traffic scenarios including modern attack behaviors. The dataset enables evaluation of detection models under practical network conditions and supports behavioral learning diversity within IS-HAITI experiments. [13]

CHAPTER 4 PROJECT DESCRIPTION

The IS-HAITI (Intelligent Smart Hybrid AI Threat Intelligence) system is designed as a hybrid cybersecurity analysis framework capable of detecting malicious activities by combining behavioral learning, structural reasoning, rule-based intelligence, and explainable analysis.

The system analyzes cyber threats at the service and communication level rather than performing simple record classification. Network traffic, communication content, and extracted threat indicators are processed through multiple analytical components working together to generate interpretable threat intelligence outputs.

4.1 System Architecture

The implemented architecture consists of four primary components:

1. **Behavioral Learning Module (Transformer Models)**
2. **Graph Intelligence Module (GNN Analysis)**
3. **Rule-Based Threat Detection**
4. **Explainable Threat Intelligence System (X-TIS)**

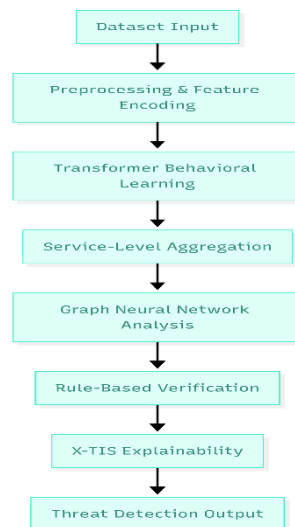


Fig 1. System Architecture

4.2 Dataset Preparation and Processing

Datasets Used:

- NSL-KDD Dataset – service and protocol based intrusion features.
- CICIDS2017 Dataset – modern enterprise network traffic.
- UNSW-NB15 Dataset – realistic contemporary attack scenarios.

Each dataset undergoes preprocessing including:

- Removal of missing values,
- Categorical feature encoding,
- Normalization of numerical attributes,
- Dataset splitting into training, validation, and testing sets,
- Tensor generation for deep learning training.

4.3 Threat Detection Capability

The system demonstrates capability in identifying multiple cyber threat categories including:

- spam and phishing communications,
- malicious links and credential harvesting websites,
- port scanning and network reconnaissance activity,
- brute-force authentication attempts,
- service enumeration behavior,
- IRC-based botnet communication patterns.

The modular design enables extension toward additional attack categories such as lateral movement, command-and-control communication, data exfiltration behavior, malware persistence activity, and multi-stage coordinated attack campaigns.

CHAPTER 5 REQUIREMENTS

The IS-HAITI system is implemented as a hybrid cyber threat intelligence platform consisting of multiple analytical components including graph-based service risk detection, Transformer-based behavioral learning, fusion modeling, content threat detection, malicious URL analysis, and explainable intelligence generation.

The system requirements are defined to support data preprocessing, deep learning model training, graph computation, threat detection workflows, and explainability report generation.

5.1 Functional Requirements

Functional requirements describe the operations that the system must perform to achieve cyber threat analysis.

5.1.1 Network Data Processing

The system shall:

- Reprocess intrusion detection datasets (KDD, CICIDS2017, UNSW-NB15),
- Convert raw records into structured tensors,
- Generate training, validation, and testing splits,
- Store dataset metadata for reproducibility.

5.1.2 Transformer Training Module

The system shall support behavioral learning using a GPU-enabled Tabular Transformer pipeline.

Required capabilities include:

- Training Transformer models using tabular network features,
- Generating model checkpoints,
- Exporting validation and testing metrics,
- Performing inference for attack probability estimation.

5.1.3 Content-Based Threat Detection (Category A)

The system shall detect phishing and spam threats using textual datasets.

Datasets used:

- SMSSpamCollection
- spam.csv
- easy_ham email corpus

System capabilities:

- analyze message content,
- detect phishing and scam patterns,
- identify malicious text behavior,
- generate explainable detection reports.

Outputs include:

- detection CSV/JSON files,
- X-TIS explanation reports,
- human-readable HTML summaries.

5.1.4 Malicious URL and Website Detection (Category B)

The system shall analyze URLs extracted from communication datasets.

Detection mechanisms include:

- lexical URL analysis,
- suspicious domain identification,
- phishing keyword detection,
- IP-based hostname analysis,
- URL shortener detection,
- redirect behavior analysis.

Generated outputs:

- malicious URL detection reports,
- explanation fields describing threat reasoning,
- HTML visualization reports.

5.2 Non-Functional Requirements

5.2.1 Performance Requirements

- **Scalability:** The system architecture should support processing of larger intrusion detection datasets and additional threat intelligence sources without major redesign. The modular pipeline must allow integration of new datasets, models, and detection modules in future extensions.
- **Reliability:** The system should generate consistent detection results when executed multiple times on similar input data. Model training, inference, and fusion processes must maintain stable performance across different datasets
- **Usability:** Detection outputs should be easily interpretable through structured CSV/JSON reports and human-readable explainability summaries (X-TIS reports), enabling understanding even for users without deep machine learning expertise.

- **Performance:** The system should provide reasonable training and inference time using GPU acceleration and optimized preprocessing pipelines, ensuring efficient execution during experimentation and testing.
- **Maintainability:** The codebase should follow modular design principles, allowing independent modification of preprocessing, Transformer training, GNN fusion, detection modules, and explainability components without affecting the entire system.
- **Security:** The system should follow safe data handling practices, ensuring datasets are processed locally without unauthorized exposure, and all model outputs are generated within controlled experimental environments.

5.3 Software Requirements

- **Operating System:** Windows 10 / Linux / macOS
- **Programming Language:** Python (Version 3.x)
- **AI/Machine Learning Frameworks:**
 - PyTorch
 - TensorFlow (optional)
- **Supporting Libraries:**
 - Scikit-learn
 - NumPy
 - Pandas
 - NetworkX / PyTorch Geometric (for graph modelling)
- **Visualization Tools:**
 - Matplotlib
 - Seaborn
- **Development Environments:**
 - Jupyter Notebook
 - VS Code or any Python IDE

5.4 Hardware Requirements

- A computer system with a minimum of **Intel Core i5 processor or equivalent**.
- **8 GB RAM** or higher for smooth execution of machine learning models.
- At least **500 GB of storage** for storing datasets, trained models, and project files.
- A **GPU** (such as NVIDIA GPU with CUDA support) is recommended for faster
- model training, though the system can also run on CPU for basic experiments.
- Stable **internet connection** for accessing datasets, libraries, and research resources.

CHAPTER 6 METHDOLOGY

This chapter explains how the IS-HAITI system processes user-provided network data and detects cyber threats using different analysis stages.

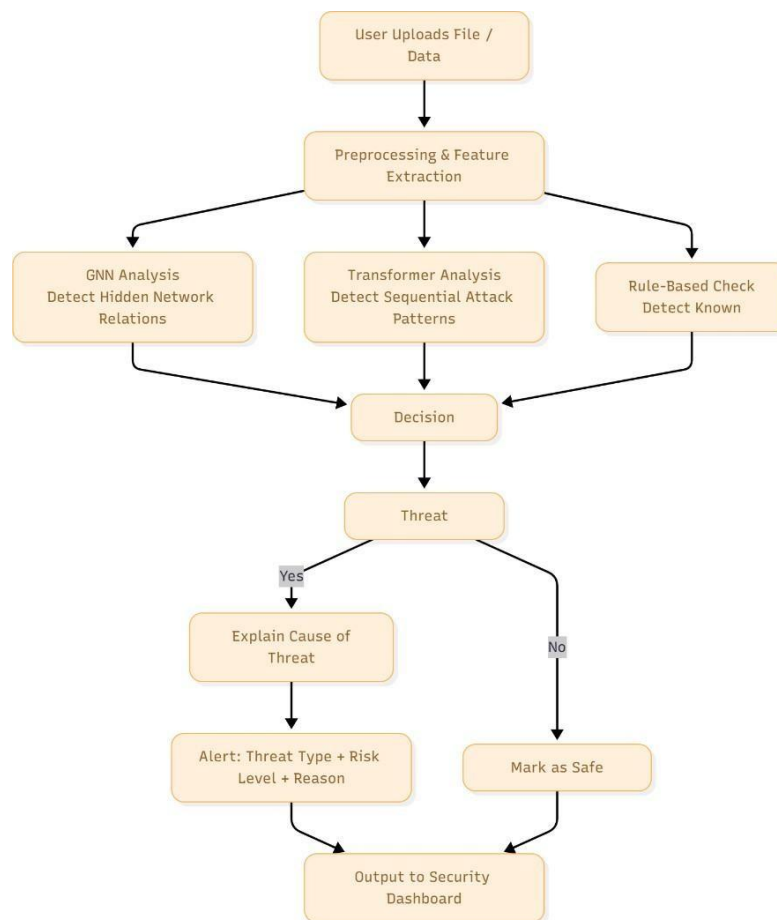


Fig 1: High-Level Workflow of the IS-HAITI Threat Detection System

Figure 1: illustrates the overall working flow of the proposed IS-HAITI threat detection system. The process begins when a user uploads network traffic data or log files into the system. This data is first passed through a preprocessing stage where it is cleaned, normalized, and converted into a suitable format for analysis.

6.1 User Data Input

The process begins when the user provides input data to the system.

- The user uploads network traffic data or system log files.
- Input data can be in the form of:
 - CSV files
 - Network flow logs
- This data acts as the initial source for all further processing and analysis.

6.2 Data Preprocessing and Feature Extraction

After uploading, the data undergoes preprocessing to make it suitable for model analysis.

Preprocessing Steps:

Removal of missing and duplicate values

- Normalization of numerical features
- Conversion of categorical features into numerical format
- Selection of only relevant network features

Feature Extraction:

The following important features are extracted:

- Flow duration
- Packet size
- Protocol type
- Connection frequency
- Source and destination information

The processed data is then separated into two formats:

- One for **graph-based analysis**
- One for **time-based sequence analysis**

6.3 Transformer-Based Flow-Level Threat Analysis

A GPU-enabled Tabular Transformer model is used as the first detection layer.

Transformer model is trained using multiple intrusion detection datasets including:

- UNSW-NB15
- CICIDS2017
- KDD Dataset

6.4 Service-Level Graph Construction

This stage analyzes how network behavior changes over time.

- Each service (FTP, IRC, Telnet, HTTP, etc.) is treated as a node.
- Communication relationships form graph edges.
- Aggregated Transformer probabilities become node features.

6.5 Explainability using X-TIS

The Explainability and Transparency Intelligence System (X-TIS) provides interpretable results for detected threats.

For each flagged entity, the system generates:

- important contributing features
- service neighborhood context
- human-readable explanation
- visualization outputs

CHAPTER 7 EXPERIMENTATION

7.1. Part 1 experimentation:

7.1.1 : Overview

This phase transforms raw network traffic flows into a structured graph representation suitable for threat detection. The pipeline consists of four main stages:

1. **Network Flow Extraction** (source/destination IPs, ports, protocols)
2. **Statistical Feature Aggregation** (packet counts, byte volumes, temporal patterns)
3. **Graph Construction** (services as nodes, communication patterns as edges)
4. **Node Classification Preparation** (binary classification: safe vs. attack-linked)

This workflow was applied consistently to the NSL-KDD and CICIDS datasets to ensure reproducibility across different network attack scenarios.

7.1.2 Data Preprocessing Pipeline

Flow-Level Extraction

- Raw network traffic from PCAP files and CSV datasets is parsed using network analysis tools (Wireshark, pyshark, pandas-compatible parsers)
- Each flow is characterized by:
 - Source IP and Port
 - Destination IP and Port
 - Protocol (TCP, UDP, ICMP)
 - Packet count, total bytes transferred
 - Flow duration (start/end timestamps)
 - Traffic direction (bidirectional aggregation)

Service Identification and Node Creation

- Destination ports are mapped to well-known services (HTTP, SSH, FTP, DNS, etc.)
- Each unique service becomes a **node** in the network graph
- Nodes store aggregated features:
 - avg_packet_size – Mean bytes per packet

- flow_count – Total number of flows to this service.
- total_bytes – Sum of bytes across all flows
- duration_stats – Mean, std of flow durations
- protocol_distribution – Ratio of TCP/UDP/ICMP flows
- attack_probability – Binary label from dataset (0=safe, 1=attack)

Temporal Feature Extraction

For time-aware analysis, the following features are computed per service:

- **Burst Detection:** Time intervals between flows; high frequency $\approx$ suspicious
- **Temporal Concentration:** % of activity in peak hours vs. off-peak hours
- **Flow Seasonality:** Consistency of traffic patterns across time windows

Statistical Feature Aggregation

For each service node, the following 12-dimensional feature vector is constructed:

- avg_packet_size – Mean packet size (bytes)
- flow_count – Number of unique flows
- total_bytes – Total data transferred
- mean_duration – Average flow duration
- std_duration – Std dev of flow durations
- tcp_ratio – Fraction of TCP flows
- udp_ratio – Fraction of UDP flows
- icmp_ratio – Fraction of ICMP flows
- unique_sources – Distinct source IPs connecting to this service
- unique_destinations – Distinct destination Ips
- burst_rate – Flows per second (temporal density)
- protocol_entropy – Entropy across protocol types

7.1.3 Graph Construction

Edge Definition

Edges connect services that communicate directly in the network:

- **Directed edges** represent communication direction (client $\rightarrow$ server convention)
- **Edge weight** = frequency of inter-service communication
- **Self-loops** are included for services with internal traffic patterns

Example: If HTTP service (port 80) receives 50 flows from SSH service (port 22), an edge is created with weight = 50.

Graph Representation

- **Adjacency Matrix** $A \in \mathbb{R}^{(N \times N)}$ where N = number of unique services
- **Feature Matrix** $X \in \mathbb{R}^{(N \times 12)}$ where each row is a service's feature vector
- **Label Vector** $y \in \{0,1\}^N$ indicating safe (0) or attack-linked (1) services

The complete graph $G = (V, E, X, y)$ is stored in PyTorch Geometric format for efficient GNN processing.

7.1.4 Dataset Normalization and Balancing

Feature Scaling

All 12 features are scaled to $[0, 1]$ range using Min-Max normalization:

- $x_{\text{scaled}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}})$
- Per-feature normalization ensures equal contribution regardless of magnitude differences

Class Balancing

Original dataset exhibited significant class imbalance (90% safe, 10% attack-linked):

- **Oversampling minority class:** Attack-linked services up-sampled using SMOTE (Synthetic Minority Over-sampling Technique)
- **Sampling strategy:** Balanced to 60% safe / 40% attack-linked ratio during training
- **Test split:** Kept at original imbalance ratio (90/10) for realistic evaluation

7.1.5 Training Configuration

- **Optimizer:** Adam (Learning Rate = $1e-3$, Weight Decay = $5e-5$)
- **Loss Function:** Binary Cross-Entropy with Logits (BCEWithLogitsLoss)
- **Epochs:** 100 with Early Stopping (patience = 15 epochs)
- **Device:** GPU (CUDA) when available; CPU fallback
- **Training Output:** 70% / 15% / 15% stratified split
- **Output :** Per-node predicted probability of being attack-linked

7.1.6 Output

The final output of Part 1 includes:

- Normalized feature matrix and adjacency matrix in PyTorch Geometric format
- Service-level labels with attack probability scores
- Dataset statistics: node count, edge density, feature distributions
- Diagnostic plots: feature correlation heatmaps, class distribution before/after balancing

7.2. Phase 2: Graph Neural Network Training and Attack Detection

7.2.1 Overview

This phase focuses on training multiple GNN architectures to classify network services as safe or attack-linked based on their local topology and feature patterns. The objective is to identify compromised or suspicious services that exhibit abnormal communication behavior.

Target classes: **Safe (0)** and **Attack-Linked (1)**

7.2.2 GNN Architecture Design

Graph Convolutional Network (GCN)

- **Layer Structure:** Input $\rightarrow$ GCN(64) $\rightarrow$ GCN(32) $\rightarrow$ MLP(16) $\rightarrow$ Output(1)
- **Activation:** ReLU between layers, Sigmoid on output
- **Mechanism:** Aggregates features from 1-hop neighborhood; effective for local patterns
- **Invalid Advantage:** Lightweight, interpretable, efficient on sparse graphs

Graph Attention Network (GAT)

- **Layer Structure:** Input $\rightarrow$ SAGE(64, mean aggregation) $\rightarrow$ SAGE(32, mean aggregation) $\rightarrow$ MLP(16) $\rightarrow$ Output(1)
- **Aggregation Function:** Mean pooling over neighbor features
- **Sampling Strategy:** Fixed-size neighborhood sampling (sample_size = [25, 10] per layer)
- **Advantage:** Scalable to large graphs; effective for inductive learning on unseen services

Hybrid GNN Ensemble

- Combines predictions from GCN + GAT + GraphSAGE with weighted voting
- Weights learned on validation set to maximize F1-score
- Ensemble consensus reduces false positives and improves robustness

7.2.3 Hyperparameter Tuning Strategy

Systematic Grid Search

- **Hidden Dimensions:** {32, 64, 128}
- **Number of Layers:** {2, 3, 4}
- **Small Dropout Rate:** {0.0, 0.3, 0.5}
- **Learning Rate:** {1e-4, 1e-3, 1e-2}

Best configuration identified:

- Hidden Dim: 64
- Layers: 3
- Dropout: 0.3
- Learning Rate: 1e-3

Early Stopping Criteria

- Monitor validation loss every 5 epochs
- Stop if no improvement for 15 consecutive epochs
- Restore best model weights before final evaluation

7.2.4 Model Training Details

Loss Function

Binary Cross-Entropy with Logits:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\sigma(z_i)) + (1-y_i) \log(1-\sigma(z_i))]$$

where z_i is the logit output and σ is the sigmoid function.

Regularization Techniques

- **Dropout:** 0.3 rate in hidden layers to prevent overfitting
- **L2 Regularization:** Weight decay = 5e-5 on all parameters
- **Graph Regularization:** Implicit through message-passing design

Training Loop

Per epoch:

- Forward pass through selected GNN architecture

- Compute loss on training set (with class weights for imbalance)
- Backward pass; optimizer step
- Evaluate on validation set; check early stopping criterion
- Log metrics: Loss, Accuracy, Precision, Recall, F1

7.2.5 Hardware and Software Specifications

- **Programming Language:** Python 3.8+
- **Deep Learning Framework:** PyTorch 1.10+
- **Graph Processing:** PyTorch Geometric (torch-geometric)
- **Data Processing:** pandas, NumPy, scikit-learn
- **Visualization:** Matplotlib, NetworkX, Plotly
- **Hardware:**
 - NVIDIA GPU (RTX 2080 Ti or equivalent) for accelerated training
 - Intel i7 / Ryzen 7 equivalent CPU
 - 32 GB RAM minimum
- **Operating System:** Linux (Ubuntu 18.04+) or Windows 10/11 with WSL2

7.2.6 Input and Output Structure

Input

Graph Representation:

- Adjacency matrix A (sparse, CSR format)
- Node feature matrix X ($N \times 12$ dense matrix)
- Node labels y (N -dimensional binary vector)

Sample Input Dimensions:

- $N = 80\text{--}150$ unique services (varies by dataset)
- 12 hand-crafted features per node
- Edge density: 5–15% (sparse graph typical)

Output

Per Service Node:

- **Predicted Probability:** $P(\text{attack}) \in [0, 1]$
- **Classification:** 1 if $P(\text{attack}) > 0.5$, else 0
- **Confidence Score:** $|P(\text{attack}) - 0.5|$ (distance from decision boundary)

Aggregate Metrics:

- Confusion matrix (service-level classification distribution)

- Precision, Recall, F1-score, ROC-AUC on test set
- Service-wise predictions exported to CSV

7.2.7 Evaluation Metrics

Classification Metrics

- **Accuracy:** $(TP + TN) / (TP + TN + FP + FN)$
- **Precision:** $TP / (TP + FP)$ – Fraction of detected attacks that are true
- **Recall:** $TP / (TP + FN)$ – Fraction of actual attacks detected
- **F1-Score:** $2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$
- **ROC-AUC:** Area Under the ROC curve; tolerance for probability thresholds

Graph-Specific Metrics

- **Community Detection:** Modularity of detected attack clusters
- **Centrality Correlation:** Betweenness/degree centrality vs. attack probability
- **Subgraph Cohesion:** Density of attack-linked service clusters

7.3. Phase 3: Attack Pattern Discovery and Threat Categorization

7.3.1 Subgraph Extraction

Attack-linked services identified by the GNN are grouped into threat clusters:

- **Connected Component Analysis:** Find connected subgraphs of attack services
- **K-Hop Expansion:** Include neighbors of attack services to identify attack propagation
- **Subgraph Visualization:** Generate network diagrams highlighting threat paths

7.3.2 Risk Scoring

Services are ranked by attack probability and integrated risk:

$$\text{Risk Score} = \alpha \cdot P(\text{attack}) + \beta \cdot \text{Centrality} + \gamma \cdot \text{Cluster Size}$$

- $\alpha = 0.5$ – Attack probability weight
- $\beta = 0.3$ – Network centrality weight (how many other services connect)
- $\gamma = 0.2$ – Attack cluster size weight
- Ranking identifies top-N services for prioritized investigation